

Memory Management Virtual Memory

Mata Kuliah Sistem Operasi

Dosen Pengampu: Triawan Adi Cahyanto, M.Kom

Program Studi Teknik Informatika

Universitas Muhammadiyah Jember

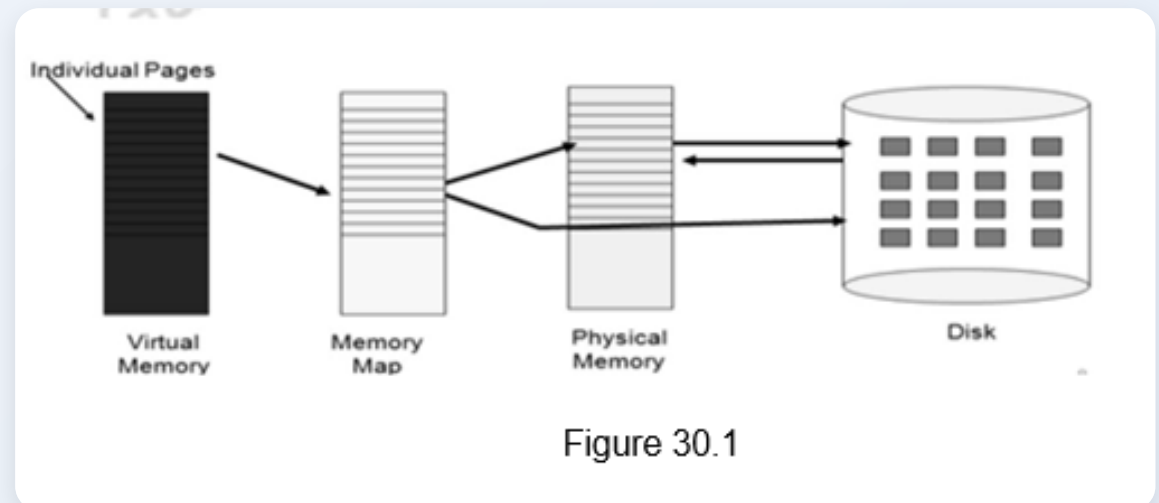
Background - Virtual Memory

🔧 Konsep Dasar Memori Virtual

- **Memori virtual** - teknik manajemen yang memberikan ilusi memori fisik lebih besar
- Menggunakan **disk** sebagai ekstensi memori fisik (RAM)
- Memisahkan **alamat logis** dari **alamat fisik** yang ada di RAM

! Mengapa Memori Virtual Diperlukan?

- Memungkinkan program lebih besar dari **RAM fisik**
- Meningkatkan **multiprogramming**- lebih banyak proses berjalan bersama
- Menyediakan **proteksi memori** antar proses
- Memudahkan **sharing kode** dan data antar proses



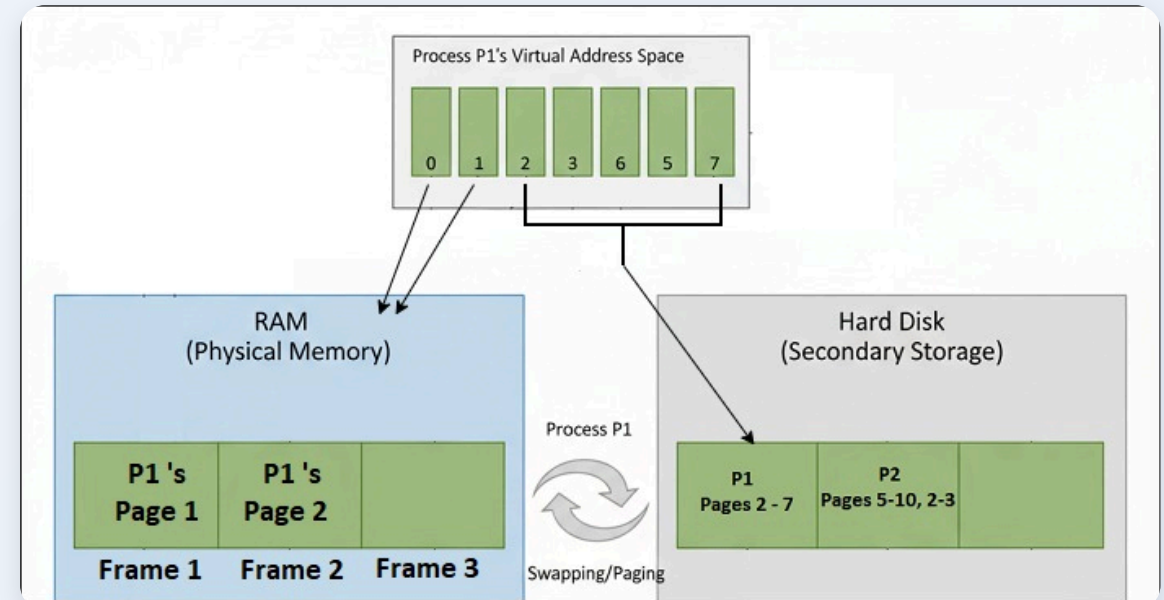
Background - Cara Kerja Memori Virtual

⚙️ Cara Kerja Memori Virtual

- Program dibagi menjadi **halaman (pages)** dengan ukuran tetap
- Memori fisik dibagi **frame** dengan ukuran sama dengan halaman
- **Page table** memetakan alamat virtual ke alamat fisik
- Halaman yang tidak digunakan disimpan di **disk**

👍 Keuntungan Memori Virtual

- **Efisiensi memori** - hanya bagian yang diperlukan dimuat ke RAM
- **Proteksi memori**- setiap proses memiliki ruang alamat terpisah
- **Sharing memori**- memungkinkan sharing kode dan data
- **Program lebih besar** dari memori fisik dapat dijalankan



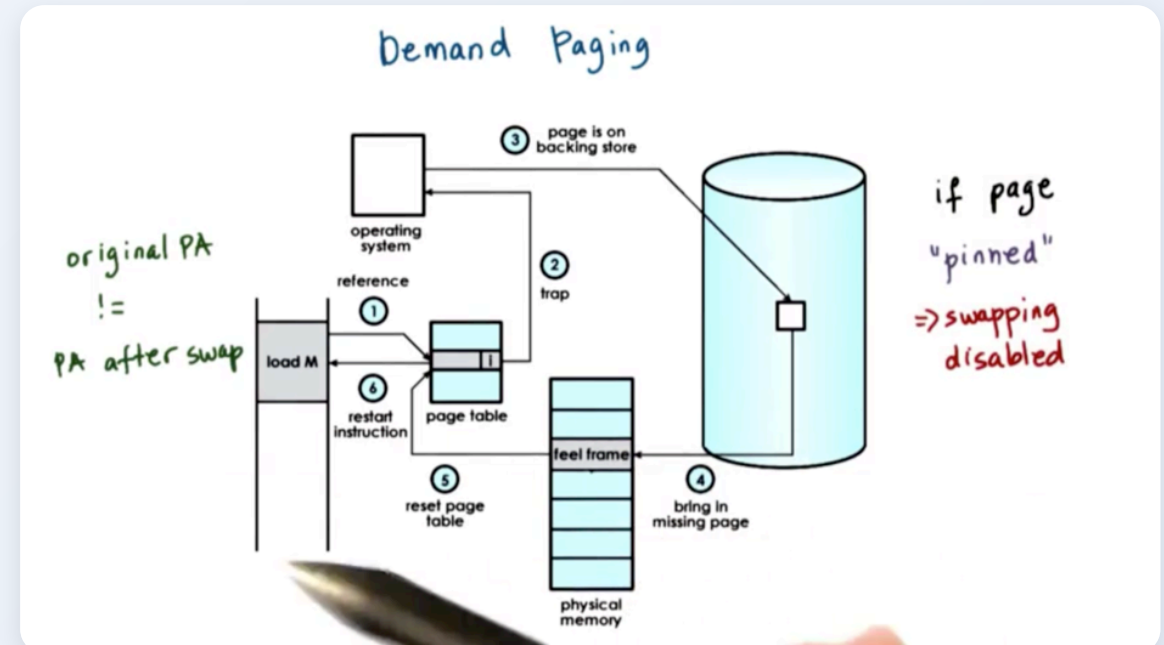
Demand Paging - Konsep Dasar

Definisi Demand Paging

- Teknik manajemen memori yang **memuat halaman** ke memori fisik **dibutuhkan** hanya saat
- Menghemat memori dengan **tidak memuat** seluruh program sekaligus
- Bagian dari implementasi **memori virtual**

Cara Kerja Demand Paging

- Program memulai eksekusi tanpa halaman di memori
- Saat akses halaman yang belum dimuat → **page fault**
- OS menemukan halaman di disk dan memuat ke memori
- Program melanjutkan eksekusi setelah halaman tersedia



Demand Paging - Proses dan Keuntungan

↻ Proses Demand Paging

- CPU mencoba mengakses halaman yang tidak ada di memori
- MMU mendeteksi **page fault** dan menginformasikan OS
- OS memeriksa validitas alamat di **page table**
- OS menemukan halaman di disk dan memuat ke **frame kosong**
- Page table diperbarui dan instruksi dijalankan kembali

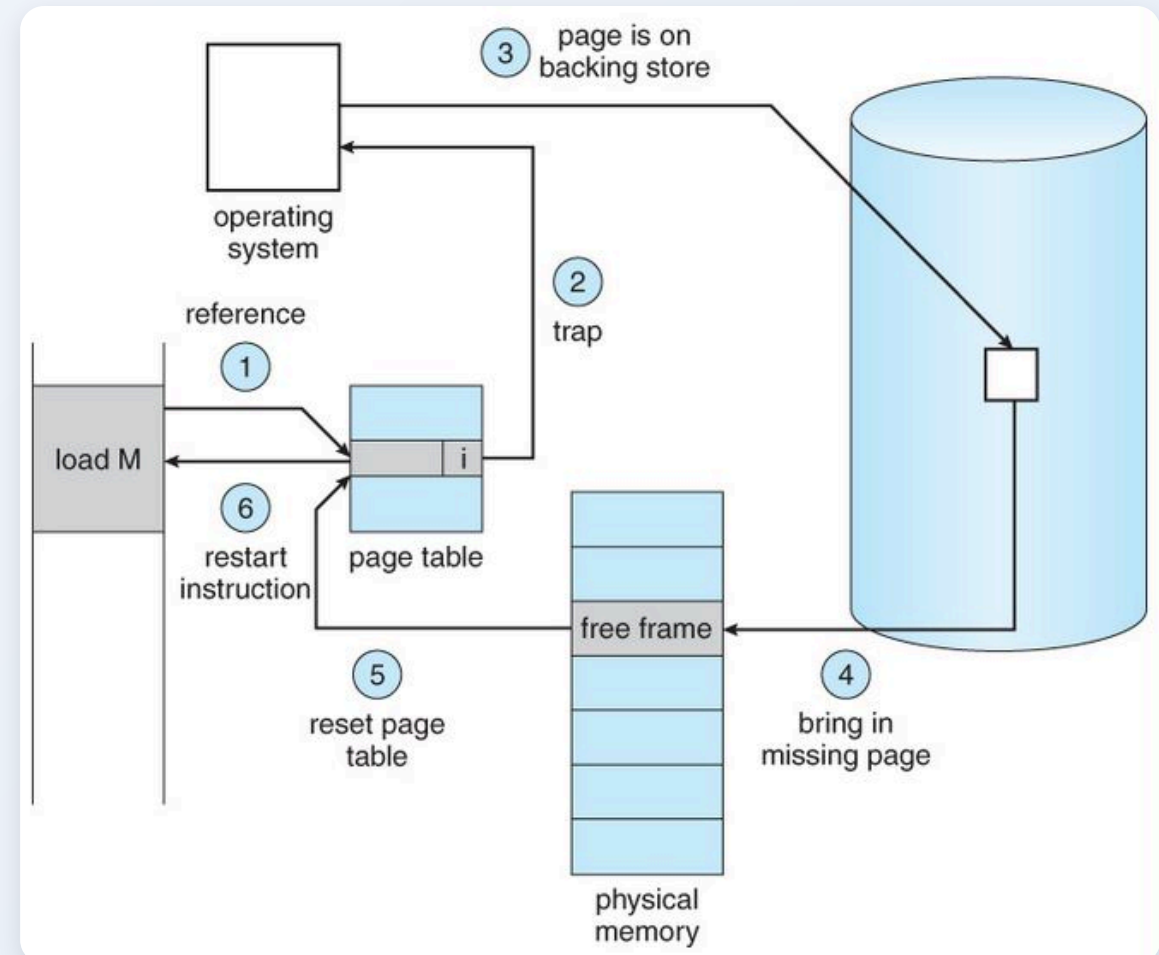
↔ Keuntungan dan Kerugian

👍 Keuntungan

- ✓ Menghemat memori fisik
- ✓ Waktu startup lebih cepat
- ✓ Program lebih besar dari RAM dapat dijalankan

👎 Kerugian

- ✗ Page fault meningkatkan overhead
- ✗ Kompleksitas implementasi lebih tinggi
- ✗ Performa tergantung kecepatan disk



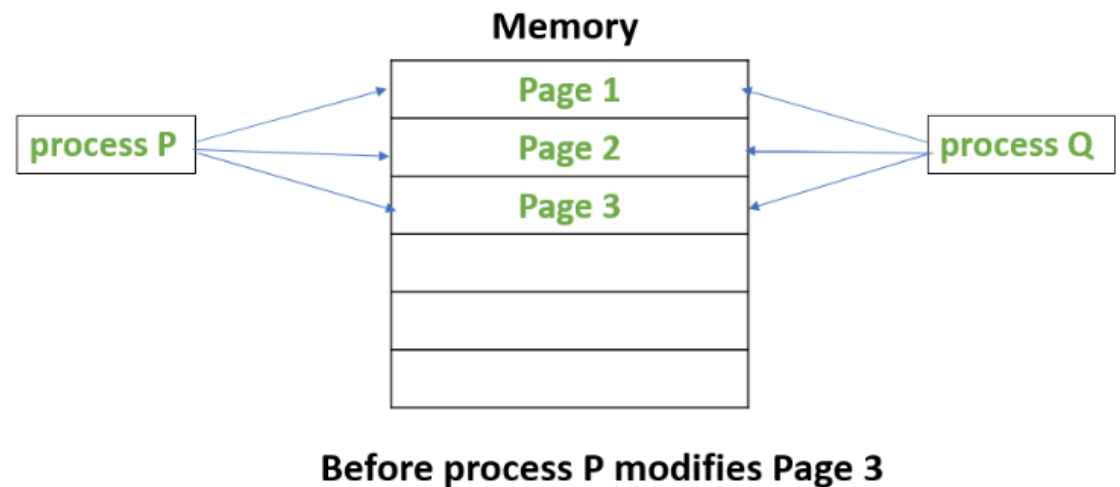
Copy-on-Write - Konsep Dasar

Konsep Copy-on-Write (CoW)

- Teknik manajemen sumber daya yang **menunda penyalinan** data sampai benar-benar diperlukan
- Memungkinkan **sharing memori** antar proses tanpa duplikasi awal
- Sering digunakan dalam implementasi **fork()** di sistem operasi

Cara Kerja Copy-on-Write

- Proses induk dan anak **berbagi halaman** memori yang sama
- Halaman ditandai sebagai **read-only** untuk kedua proses
- Saat salah satu proses mencoba menulis → **page fault**
- OS membuat salinan halaman untuk proses yang menulis
- Proses lain tetap menggunakan halaman asli



Copy-on-Write - Implementasi dan Keuntungan

📁 Implementasi Copy-on-Write

- System call `fork()` di Unix/Linux
- Memory-mapped files untuk sharing data
- Virtualization dan containerization
- Version control systems (Git)

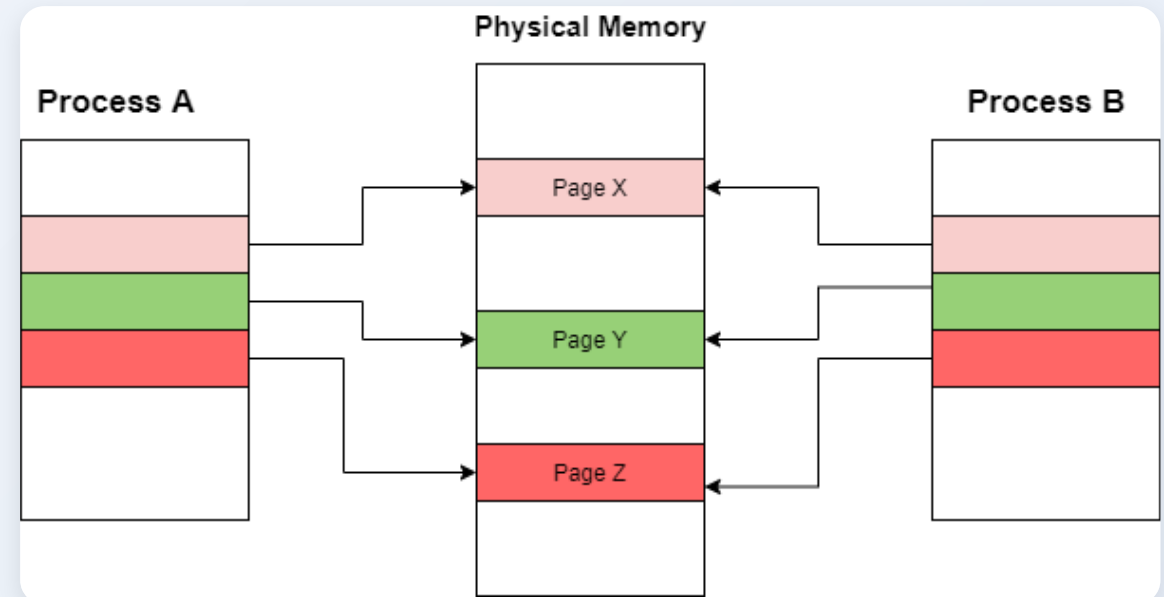
↔️ Keuntungan dan Kerugian

👍 Keuntungan

- ✓ Menghemat memori fisik
- ✓ Proses `fork()` lebih cepat
- ✓ Mengurangi I/O disk

👎 Kerugian

- ✗ Overhead saat penulisan pertama
- ✗ Kompleksitas implementasi
- ✗ Fragmentasi memori



Page Replacement - Konsep Dasar

↔ Konsep Page Replacement

- Proses **mengganti halaman** di memori fisik saat terjadi page fault
- Memungkinkan **memori virtual** lebih besar dari memori fisik
- Bagian penting dari sistem **demand paging**

! Mengapa Diperlukan?

- Saat **tidak ada frame kosong** saat terjadi page fault
- Untuk **mengoptimalkan** penggunaan memori fisik
- Mengurangi **frekuensi page fault** dan meningkatkan performa
- Membuat sistem dapat menjalankan **program lebih besar** dari RAM

Page
reference

1, 3, 0, 3, 5, 6, 3

1	3	0	3	5	6	3
		0	0	0	0	3
	3	3	3	3	6	6
1	1	1	1	5	5	5
Miss	Miss	Miss	Hit	Miss	Miss	Miss

Total Page Fault = 6

Page Replacement - Algoritma

↔ Algoritma Page Replacement

- **FIFO**- First In First Out
- **LRU**- Least Recently Used
- **Optimal**- Mengganti halaman yang akan digunakan paling lama
- **LFU**- Least Frequently Used



FIFO

Mengganti halaman yang paling lama berada di memori



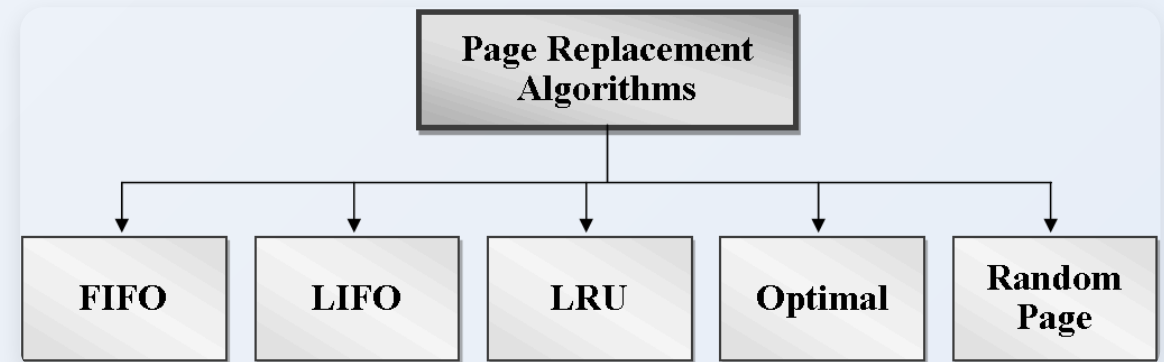
LRU

Mengganti halaman yang paling lama tidak digunakan



Optimal

Mengganti halaman yang akan digunakan paling lama di masa depan



Page Replacement - Perbandingan Algoritma

Perbandingan Algoritma

Algoritma	Kelebihan	Kekurangan
FIFO	Sederhana, mudah diimplementasi	Belady's anomaly, tidak optimal
LRU	Performa baik, intuitif	Memerlukan hardware tambahan
Optimal	Page fault minimum	Tidak praktis, perlu prediksi masa depan

👍 Kriteria Pemilihan

- ✓ Jumlah page fault minimal
- ✓ Overhead implementasi rendah
- ✓ Kompatibilitas hardware

❗ Pilihan Praktis

- ✗ LRU - keseimbangan performa
- ✗ FIFO - sistem sederhana
- ✗ Clock - variasi LRU praktis

1	2	3	4	5	1	3	1	6	3	2	3
1	1	1	1	5	5	5	5	5	5	2	2
	2	2	2	2	1	1	1	1	1	1	1
		3	3	3	3	3	3	3	3	3	3
			4	4	4	4	4	6	6	6	6
M	M	M	M	M	M	H	H	M	H	M	H

M = Miss
H = Hit

Allocation of Frames - Konsep Dasar

■ Konsep Alokasi Frame

- Proses **membagi memori fisik** menjadi frame dengan ukuran tetap
- Menentukan **jumlah frame** yang dialokasikan untuk setiap proses
- Mempengaruhi **performa sistem** dan tingkat page fault

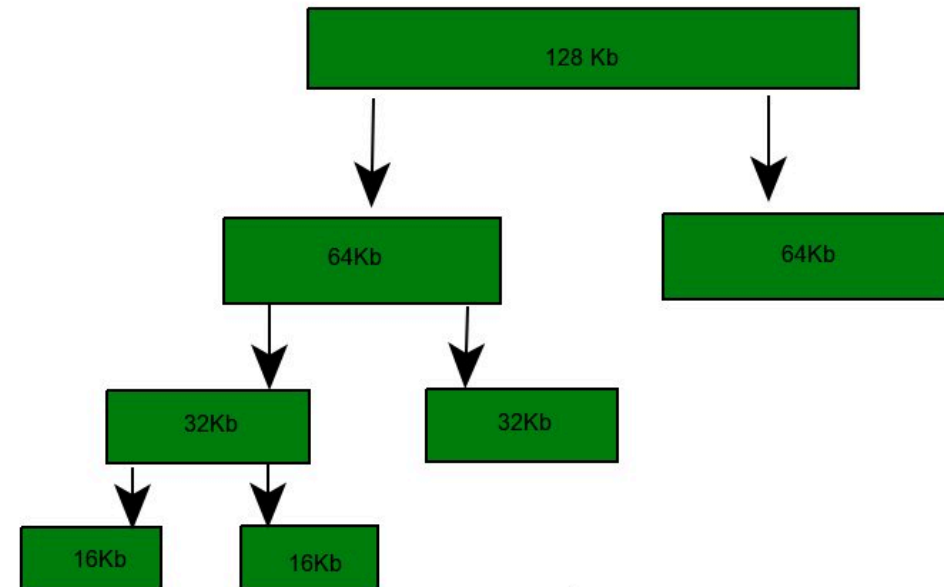
! Mengapa Diperlukan?

- Mencegah **thrashing** dengan alokasi yang tepat
- Mengoptimalkan **penggunaan memori** antar proses
- Meningkatkan **multiprogramming** dengan efisien
- Menyeimbangkan **beban sistem** antar proses

■ Algoritma Alokasi



Equal allocation, Proportional allocation, Priority allocation



Allocation of Frames - Algoritma

Algoritma Alokasi Frame



Equal Allocation

Setiap proses mendapat **jumlah frame yang sama**

Frame per proses = $\text{Total frames} / \text{Jumlah proses}$



Proportional Allocation

Frame dialokasikan **sebanding dengan ukuran proses**

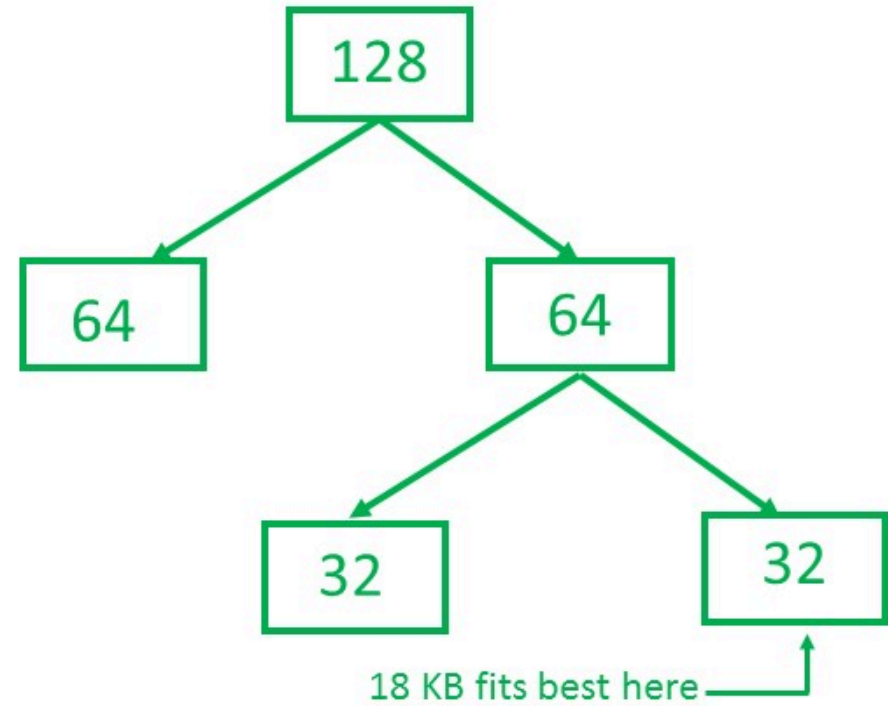
Frame untuk proses $i = \frac{\text{Size}(i)}{\sum \text{Size}} \times \text{Total frames}$



Priority Allocation

Proses dengan **prioritas lebih tinggi** mendapat lebih banyak frame

Frame untuk proses $i = \frac{\text{Prioritas}(i)}{\sum \text{Prioritas}} \times \text{Total frames}$



Thrashing - Konsep Dasar

⚠ Definisi Thrashing

- Sistem menghabiskan **waktu lebih banyak** untuk swapping daripada eksekusi
- Page fault rate **sangat tinggi** karena proses tidak memiliki cukup frame
- CPU utilization **menurun drastis** meskipun ada banyak proses aktif

⚠ Gejala Thrashing

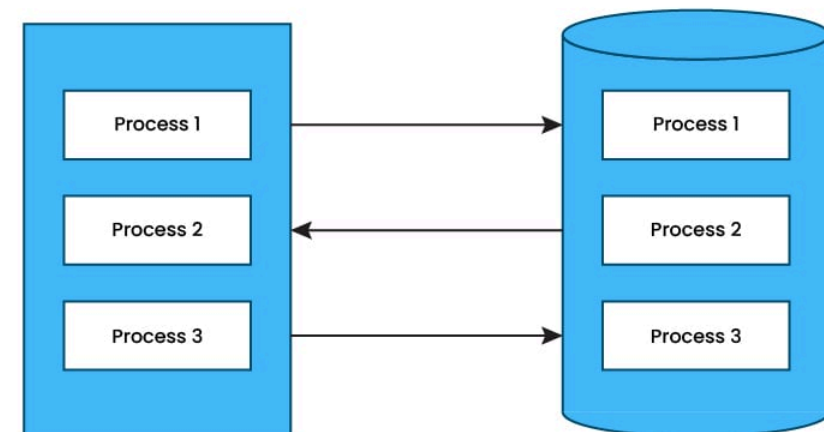
- 📉 Performa rendah
- ↔ I/O disk tinggi
- 🔧 Page fault tinggi
- 🕒 Waktu tunggu panjang

! Penyebab Thrashing

- Degree of multiprogramming terlalu tinggi
- Proses tidak memiliki **cukup frame** untuk bekerja efisien
- Working set proses **melebihi** jumlah frame yang dialokasikan
- Page replacement algorithm **tidak optimal** untuk beban kerja



THRASHING



Thrashing - Cara Mengatasi

⚙️ Working Set Model

- Model untuk menentukan **jumlah frame minimum** yang dibutuhkan proses
- Working set = **kumpulan halaman** yang sering diakses dalam periode waktu tertentu
- OS mengalokasikan frame berdasarkan **ukuran working set**

$WSS(t, \Delta) = \{\text{halaman yang diakses dalam interval } \Delta\}$

🔧 Strategi Mengatasi Thrashing



Local Replacement

Proses hanya mengganti halaman miliknya sendiri



Page Fault Frequency

Menambah frame jika fault rate tinggi



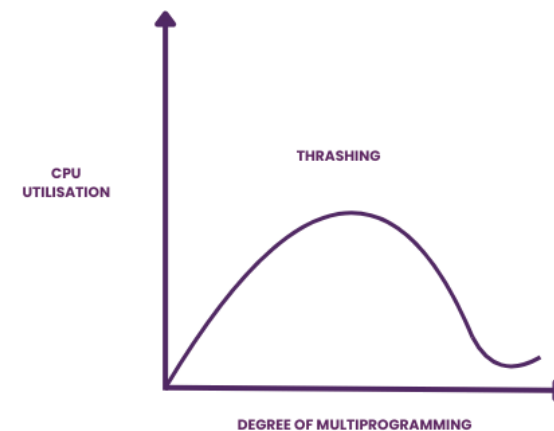
Swapping

Memindahkan proses ke disk sementara



Prioritas

Memberikan prioritas lebih tinggi pada proses penting



Memory Compression - Konsep Dasar

≡ Konsep Memory Compression

- Teknik **mengompresi halaman memori** yang jarang digunakan
- Menyimpan halaman terkompresi di **memori fisik** bukan disk
- Bagian dari **memori virtual** management

! Mengapa Diperlukan?

- Mengurangi **penggunaan memori fisik** tanpa swapping ke disk
- Meningkatkan **performa sistem** dengan mengurangi I/O disk
- Mengatasi **thrashing** dengan efisiensi memori lebih tinggi



Performa Tinggi

Akses memori terkompresi lebih cepat dari disk



Efisiensi Memori

Menghemat ruang hingga 50% atau lebih

Before memory compression



After memory compression



Inactive

Active

Free Space

Memory Compression - Implementasi

Implementasi Memory Compression

- **Windows**- Memory Manager (MM) dengan "store" buffer
- **Linux**- zswap dan zram untuk memori terkompresi
- **macOS**- Compressed Memory dengan VM Compressor



Hardware Support

CPU dengan instruksi kompresi/dekompresi



Algoritma

LZ4, Zstandard, LZO, XZ

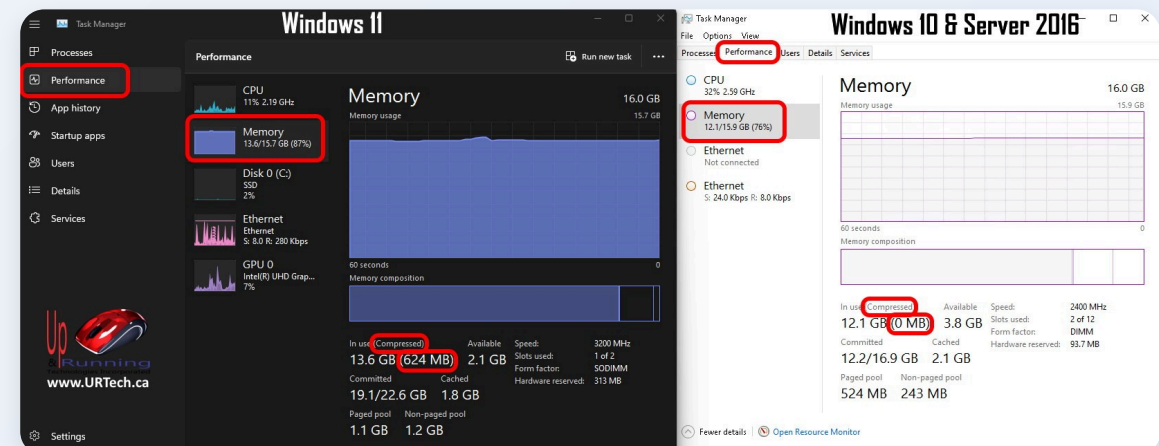
Keuntungan dan Kerugian

Keuntungan

- ✓ Mengurangi I/O disk
- ✓ Performa lebih baik dari swapping
- ✓ Menghemat konsumsi daya

Kerugian

- ✗ Overhead CPU untuk kompresi
- ✗ Kompleksitas implementasi
- ✗ Latensi akses lebih tinggi



Allocating Kernel Memory - Konsep Dasar

🔧 Konsep Alokasi Memori Kernel

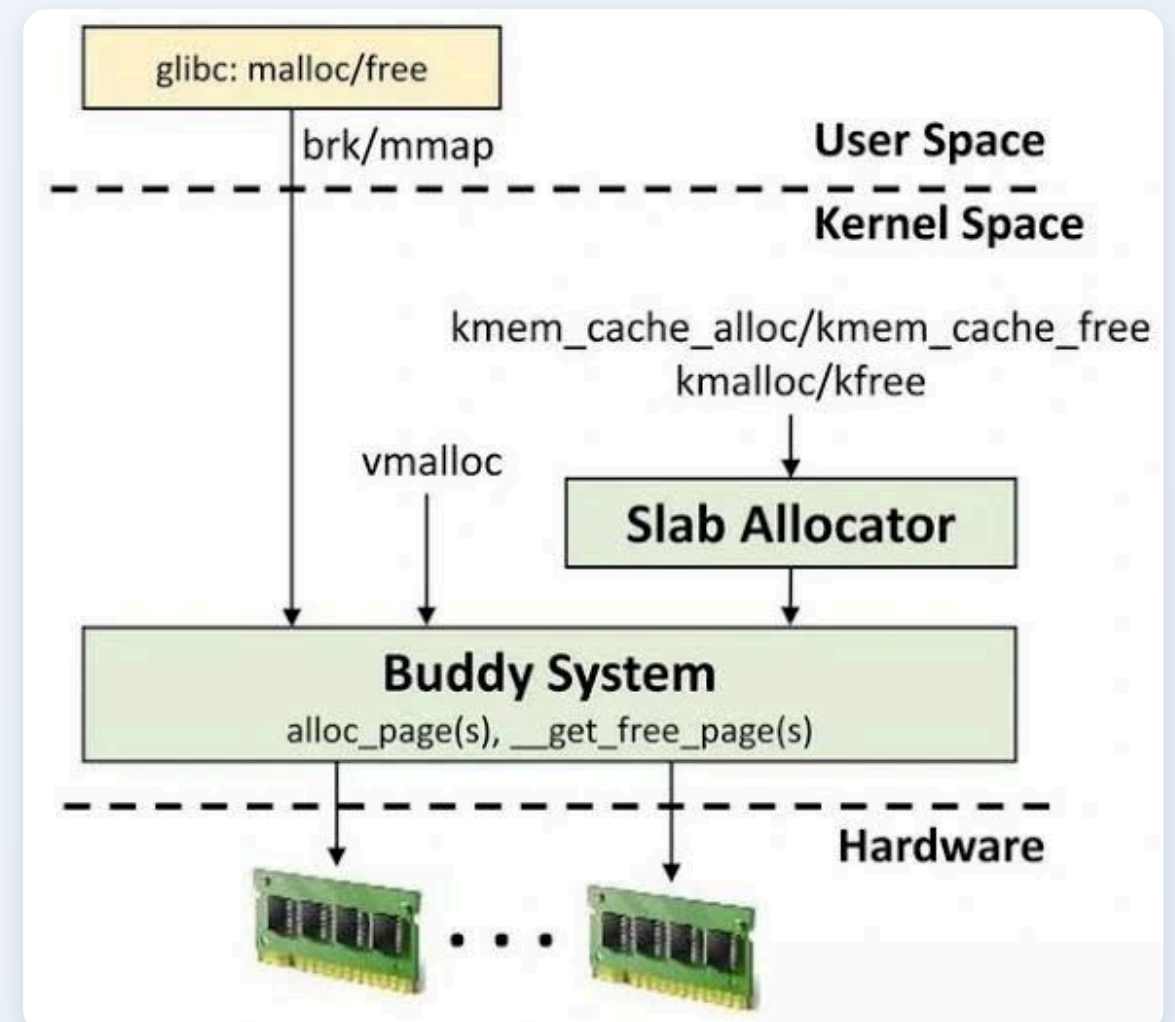
- Proses **mengalokasikan memori** untuk operasi kernel
- Memori kernel **tidak dapat dipaging** ke disk
- Menggunakan **algoritma khusus** untuk efisiensi
- Diperlukan untuk **struktur data kernel** dan buffer

↔ Kernel vs User Space

Aspek	Kernel Memory	User Memory
Paging	Tidak dapat dipaging	Dapat dipaging
Ukuran	Bervariasi	Halaman tetap

! Mengapa Diperlukan?

- **Struktur data kernel** dengan ukuran beragam
- **Performa tinggi** dengan overhead minimal
- **Efisiensi memori** untuk sumber daya terbatas
- **Fragmentasi internal** harus diminimalkan



Allocating Kernel Memory - Buddy System & Slab System

▣ Buddy System

- Memori dibagi menjadi **blok daya 2**(2^k)
- **Blok "buddy"**- pasangan blok dengan ukuran sama
- Algoritma **menggabungkan** blok-blok yang berdekatan
- Meminimalkan **fragmentasi eksternal**

👍 Kelebihan

- Algoritma sederhana
- Operasi cepat
- Fragmentasi eksternal minimal

🗨️ Kekurangan

- Fragmentasi internal tinggi
- Ukuran blok terbatas

▣ Slab System

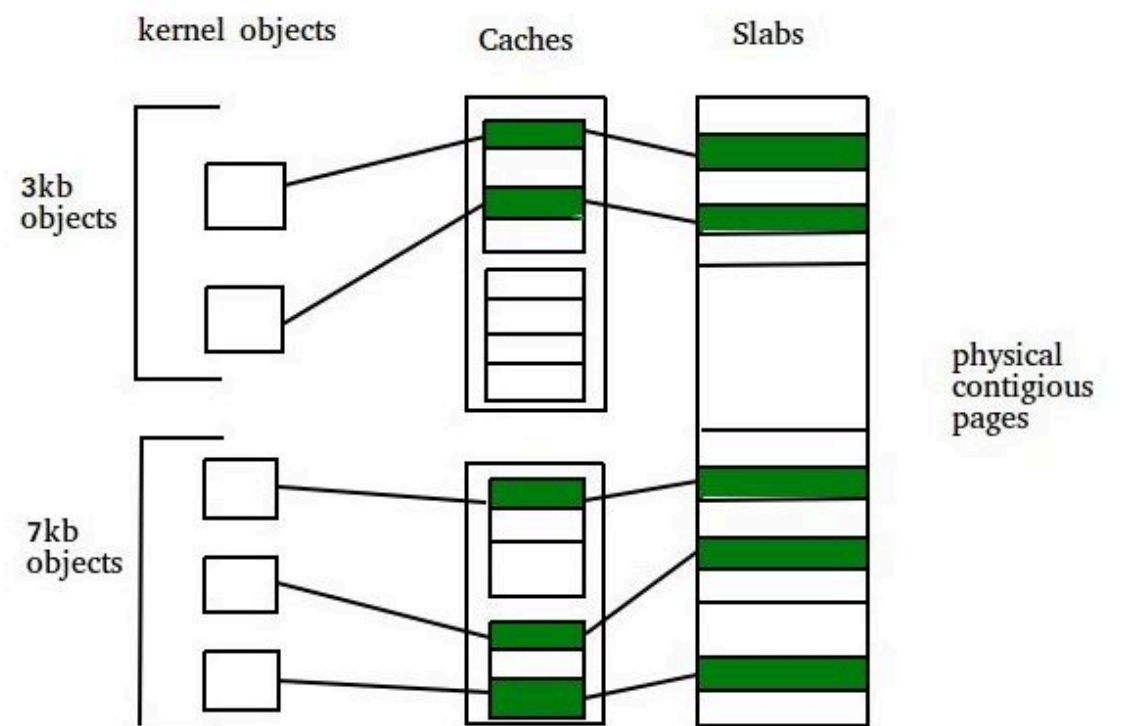
- **Cache objek** untuk tipe data kernel spesifik
- Menggunakan **slab**- blok memori berisi objek
- Objek **dipre-alokasi** dan diinisialisasi
- Objek yang tidak digunakan disimpan di **free list**

👍 Kelebihan

- Fragmentasi minimal
- Akses cepat ke objek
- Ukuran objek fleksibel

🗨️ Kekurangan

- Kompleksitas tinggi
- Memori terbuang untuk objek tidak aktif



Other Considerations & Operating-System Examples

💡 Pertimbangan Lain

- **Prepaging**- memuat halaman yang mungkin dibutuhkan
- **Page size** - pengaruh ukuran halaman terhadap performa
- **TLB reach** - jumlah halaman yang dapat diakses tanpa TLB miss
- **Program structure**- lokasi kode dan data yang optimal
- **I/O interlock**- sinkronisasi akses I/O dan memori

🖥️ Implementasi di Sistem Operasi



Windows

↔ Working Set

≡ Memory Compression

⚙ SuperFetch



Linux

🔧 Slab Allocator

≡ zswap/zram

🛑 OOM Killer



macOS

≡ Compressed Memory

📊 Dynamic Pager

📈 Memory Pressure

Summary - Memory Management Virtual Memory



Background

- **Memori virtual**- ilusi memori lebih besar
- Menggunakan **disk** sebagai ekstensi RAM
- Memisahkan **alamat logis** dan fisik



Demand Paging

- Memuat halaman **saat dibutuhkan**
- Menghemat memori **tidak** seluruh dengan **memuat** program
- Menangani **page fault** secara efisien



Copy-on-Write

- **Menunda penyalinan** data sampai diperlukan
- Memungkinkan **sharing memori** antar proses
- Optimalkan implementasi **fork()**



Page Replacement

- **Algoritma**: FIFO, LRU, Optimal
- Memilih halaman yang akan **diganti**
- Mengurangi **page fault** dan meningkatkan performa



Allocation of Frames

- **Equal, Proportional**, dan **Priority** allocation
- Menentukan **jumlah frame** untuk setiap proses
- Mencegah **thrashing** dengan alokasi optimal



Thrashing

- Sistem menghabiskan waktu untuk **swapping**
- **Working set model** untuk mencegah
- Strategi: **local replacement**, swapping



Memory Compression

- **Mengompresi halaman** yang jarang digunakan
- Menyimpan di **memori fisik** bukan disk
- Meningkatkan **performa sistem**



Kernel Memory

- **Buddy System**- blok daya 2
- **Slab System**- cache objek
- Memori kernel **tidak dapat dipaging**

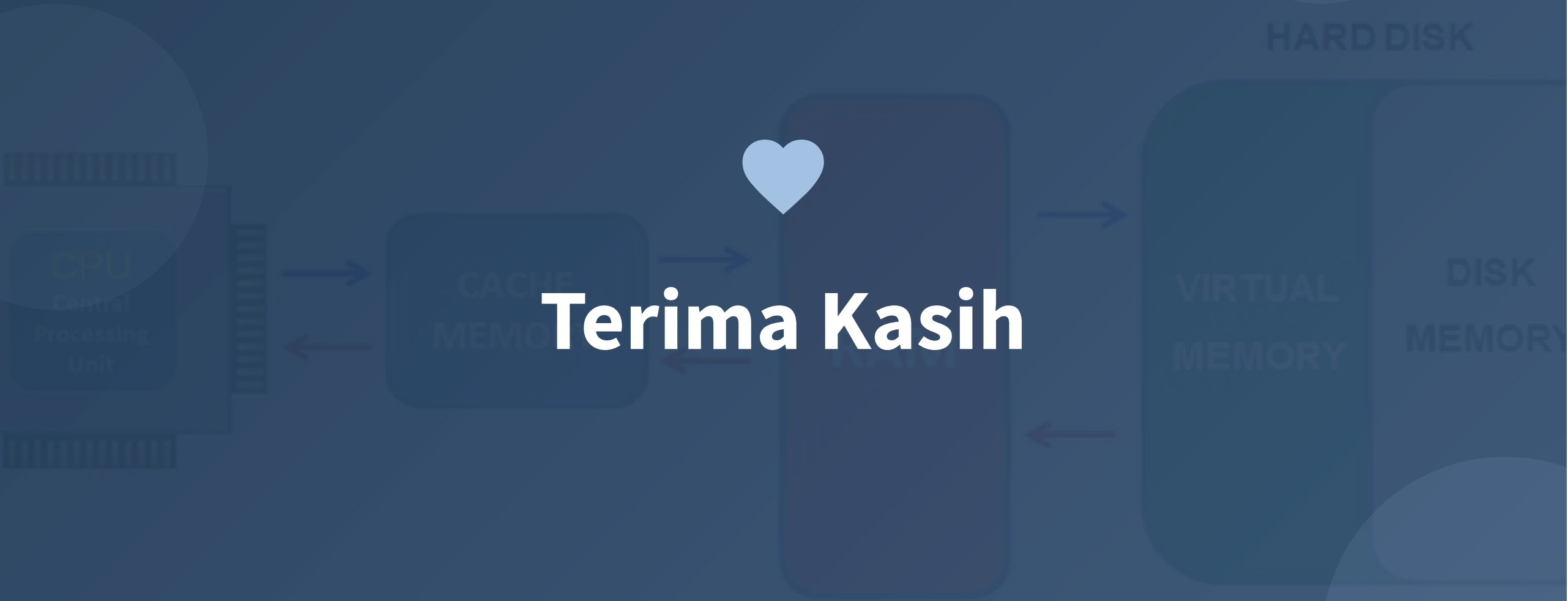


OS Examples

- **Windows**- Working Set, Memory Compression
- **Linux**- Slab Allocator, zswap/zram
- **macOS**- Compressed Memory



Memori virtual adalah teknik krusial dalam sistem operasi modern yang memungkinkan efisiensi penggunaan sumber daya, multiprogramming yang lebih baik, dan isolasi memori antar proses.



Terima Kasih